# Sistema de Moeda Estudantil

Reconhecimento acadêmico que se transforma em benefícios.

## Mérito → moedas → vantagens

Guilherme Horta Dias
Lucas Batista Duarte
Rafael Abras

Engenharia de Software · Profa. Milena Menezes Adão · 2026/2

# Do reconhecimento ao benefício

Três perfis participam de um único ciclo de mérito estudantil.

## Professor

Reconhece o aluno e registra o motivo do envio de moedas.

## Aluno

Acompanha saldo e extrato e troca moedas por vantagens.

## Empresa parceira

Publica ofertas e recebe o cupom do resgate por email.

# Como as moedas circulam

Exemplo ilustrativo de reconhecimento e resgate.

## 1.000 moedas

O professor recebe crédito a cada semestre, somado ao saldo restante.

## Reconhecimento

O professor envia 100 moedas e informa o motivo. O aluno recebe o crédito.

## Resgate

Uma vantagem de 60 moedas gera um cupom. O aluno fica com 40 moedas.

# Responsabilidades bem definidas

A interface apresenta; os serviços aplicam as regras de negócio.

**React + TypeScript**
Telas e chamadas HTTP

›

**Controllers**
HTTP, DTOs e validação

›

**Services**
Regras e transações

›

**JPA + PostgreSQL**
Entidades e persistência

Spring Boot no backend · Flyway versiona o esquema do banco.

# Identidade, saldo e histórico

Visão resumida das relações; o ER completo está na documentação.

## Identidade

Usuário vincula aluno, professor ou empresa.
Aluno e professor têm instituição.

## Moedas

Aluno e professor têm conta.
Movimentações registram créditos, envios e resgates.

## Benefícios

Empresa publica vantagem.
Resgate guarda cupom.
Notificação registra o email.

# Uma operação, um registro consistente

Saldo, movimentação e intenção de email são confirmados juntos.

## Saldo protegido

Bloqueio de contas e validação impedem débito acima do saldo.

## Sem duplicidade

Uma identificação por comando evita repetir o débito ao reenviar a operação.

## Histórico preservado

Inativação conserva cadastros e movimentações já realizadas.

# Cada perfil acessa suas funções

Autorização no backend acompanha a navegação da interface.

## Autenticação

Senhas com BCrypt. Sessão com cookie HttpOnly e proteção CSRF.

## Autorização

Aluno e empresa mantêm apenas seu próprio cadastro. Professor reconhece alunos.

## Validação

DTOs e Bean Validation. Entidades e hashes de senha não são expostos pela API.

# O resgate chega às duas partes

Aluno e empresa recebem o mesmo código de cupom.

| | |
|---|---|
| **Resgate confirmado** | Banco registra débito, cupom e notificações. |
| **Worker de email** | Processa o envio e tenta novamente após falha. |
| **Mailpit local** | Captura mensagens em localhost:8025. |

# Criar, consultar, editar e inativar

A mesma estrutura de operações atende aos dois perfis.

## Cadastro e consulta

POST cria o cadastro.
GET consulta os dados do titular autenticado.

## Atualização

PUT altera os dados.
A empresa mantém nome e email; o aluno mantém seus dados acadêmicos.

## Inativação

DELETE inativa o cadastro e encerra a sessão.
O histórico é conservado.

# O modelo orienta a implementação

Correspondências que podem ser conferidas no repositório.

## Classes do domínio

Usuario, Aluno e Conta aparecem no modelo e nas entidades do pacote cadastro.

## Componentes MVC

React → controllers → services → repositories. A separação aparece no código.

## Modelo ER

Entidades e relações acompanham as migrações SQL em db/migration.

# React: da tela à requisição

Uma alteração de cadastro percorre componentes e cliente HTTP.

## 1. Localizar a tela

Abrir CadastroForm.tsx. Identificar campos, estado do formulário e ação de envio.

## 2. Seguir o envio

Abrir api.ts. Observar os tipos de dados, a chamada HTTP e o tratamento de erro.

## 3. Observar no navegador

Editar um cadastro. Em Rede / Network, conferir a requisição, o status e a resposta.

# Spring Boot: da rota à regra

Seguir uma requisição pelo controller e pelo serviço.

## Rota HTTP

Em CadastrosController, localizar @PostMapping("/alunos") e @RequestBody.

## Validação e resposta

@Valid valida a entrada. ResponseEntity.created devolve 201 e o endereço do novo recurso.

## Regra de negócio

Em CadastrosService, localizar cadastrarAluno e acompanhar a transação.

# JPA e Flyway: do objeto à tabela

O ORM acessa dados; as migrações definem o esquema.

## Entidade JPA

Abrir Aluno.java. Localizar @Entity, identificador e relações com usuário e instituição.

## Repository

Abrir AlunoRepository. JpaRepository fornece operações básicas sobre Aluno e UUID.

## Migração SQL

Abrir V1__cadastros.sql. Comparar tabelas, chaves e restrições com as entidades.

# Docker e Mailpit: observar o email

Um ambiente local reproduzível para a demonstração.

**docker compose up -d db mailpit**

## Serviços locais

compose.yaml define PostgreSQL e Mailpit. SMTP local: porta 1025.

## Configuração

O backend usa SMTP_HOST e SMTP_PORT do ambiente. Não exibir senhas.

## Conferência

Abrir localhost:8025. Reconhecer um aluno e observar a notificação recebida.

# Percorrer o ciclo na aplicação

Aplicação: localhost:5173 · Emails: localhost:8025

| | | |
|---|---|---|
| 01 | **Aluno e empresa** | Criar, consultar e editar os dois perfis; publicar vantagem. |
| 02 | **Professor** | Entrar, escolher aluno e enviar moedas com um motivo. |
| 03 | **Aluno** | Conferir o extrato e resgatar a vantagem. |
| 04 | **Mailpit** | Abrir os emails do resgate e comparar o código do cupom. |

# Do mérito ao benefício, com histórico

Cadastro → reconhecimento → resgate → cupom → email

## Três perfis conectados por um fluxo de moedas rastreável.

MVC · PostgreSQL · transações · autorização · notificações

**Demonstração e perguntas**